AI Context Handoff & Persistent Decision Memory

1. Project Overview

Project Name

AI Context Handoff Layer

Core Idea

Modern AI coding assistants and multi-model AI environments allow users to switch between different AI models such as GPT, Claude, Gemini, Qwen, and other models.

However, when the model changes, the new model may not fully understand:

- What the previous model was working on
- Which architectural decisions were already made
- Why a particular implementation was chosen
- Which approaches were rejected
- What remains unfinished
- What constraints the project has
- What the previous model discovered while working

The proposed system solves this by creating a **persistent, model-independent memory layer** between AI models.

Instead of relying entirely on the conversation history, the system extracts important project knowledge and stores it in structured files.

Example:

AI Model A
↓
works on project
↓
decision / discovery / progress
↓
Memory Extraction
↓
Persistent Project Memory
↓
AI Model B
↓

reads relevant memory
↓
continues work

The goal is not to store the AI's private chain-of-thought.

Instead, the system stores **decisions, rationale, project state, constraints, discoveries, tasks, and other information required for another AI to continue the work correctly.**

---

2. Problem Statement

2.1 The Current Problem

AI coding assistants operate primarily around conversation context.

A typical workflow looks like:

User
↓
AI Model A
↓
Conversation
↓
AI Model A works on code

The user may then switch to another model:

User
↓
AI Model B
↓
Same project

The second model may receive some conversation history, summaries, or project files, but important information can still be lost.

For example:

Model A decides:

> PostgreSQL will be used instead of MongoDB because the application requires transactional relationships between payments, subscriptions, and users.

The actual source code may show that PostgreSQL is being used.

However, the **reason for the decision** may exist only in the previous conversation.

If Model B later sees the project and thinks:

> MongoDB would be simpler here.

it may propose or implement a change that contradicts the previous architectural decision.

---

3. The Core Problem

The fundamental problem is:

> **AI models can share source code, but they do not necessarily share the complete reasoning context, decisions, constraints, and project state required to safely continue previous work.**

This becomes more significant when:

- Switching between models
- Switching between AI providers
- Using local and cloud models together
- Context windows become full
- Long conversations are summarized
- Multiple agents work on the same repository
- Different AI coding tools are used
- An agent session ends and another starts

---

4. Goals

The system should:

Primary Goals

1. Preserve important project decisions.
2. Preserve the rationale behind decisions.
3. Preserve current project state.
4. Preserve unfinished tasks.
5. Preserve important constraints.
6. Allow different AI models to share the same memory.
7. Reduce dependence on large conversation histories.

8. Make model switching safer.
9. Work with local and cloud models.
10. Keep memory human-readable.
11. Keep memory inside the project when desired.
12. Prevent important decisions from being silently overwritten.

---

5. Non-Goals

The system should **not** attempt to:

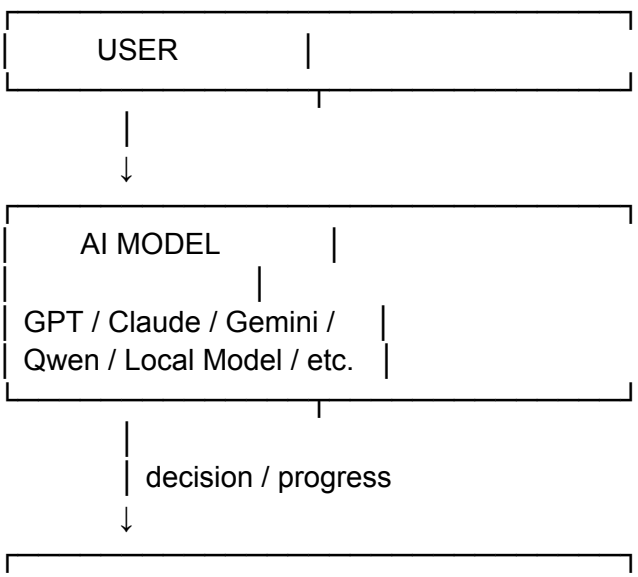
- Store private chain-of-thought.
- Record every message from the conversation.
- Save every insignificant AI response.
- Replace source control.
- Replace Git.
- Automatically modify architectural decisions without validation.
- Become a complete vector database in the first version.
- Depend on one specific AI provider.

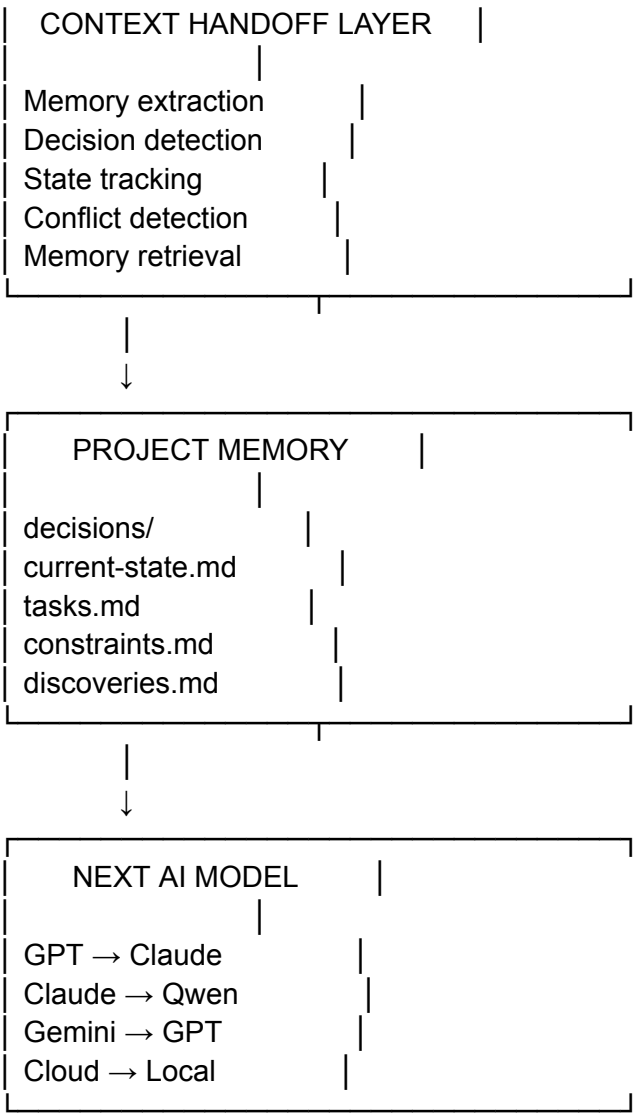
The system should store **useful project knowledge**, not the entire internal reasoning process of the model.

---

6. Core Concept

The system introduces a new layer:





---

7. Important Design Principle

Do not store "thinking"

The system should not attempt to save private chain-of-thought.

Instead, it should save a structured **decision rationale**.

Bad

I considered MongoDB, PostgreSQL and MySQL.
First I thought MongoDB was better, then I realized
transactions might be a problem...

Good

Decision:
Use PostgreSQL.

Reason:
The application requires transactional relationships
between users, payments and subscriptions.

Alternatives considered:
MongoDB

Status:
Active

This makes the memory:

- Smaller
- More reliable
- Easier to retrieve
- Easier for humans to understand
- Safer to pass between models

---

8. Main Use Cases

Use Case 1 — Switching AI Models

Claude
↓
working on authentication
↓
creates architectural decision
↓
memory saved
↓
user switches to GPT
↓
GPT reads memory
↓
continues implementation

Use Case 2 — Local + Cloud Models

Example:

Claude
↓
architecture planning

Qwen local
↓
implementation

GPT
↓
debugging

Qwen local
↓
testing

All models share:

/docs/

instead of depending completely on individual conversations.

Use Case 3 — Context Window Limit

Suppose a conversation becomes:

80,000 tokens

Instead of repeatedly passing everything:

80k conversation
+
source code
+
new request

the system can maintain:

2k–5k relevant project memory
+
relevant source files
+
current request

This reduces unnecessary context.

Use Case 4 — New AI Session

A developer closes their AI coding assistant.

The next day:

New session
↓
Read project memory
↓
Understand current state
↓
Continue work

The AI does not need the original conversation.

Use Case 5 — Multiple Agents

Example:

Agent A
Backend

Agent B
Frontend

Agent C
Testing

Agent D
Database

All agents can share the same project memory.

---

9. Memory Types

The system should initially support six major memory types.

9.1 Decisions

Permanent or semi-permanent architectural decisions.

Example:

Database → PostgreSQL
Authentication → JWT + httpOnly cookies
Frontend → Next.js
Cache → Redis

9.2 Current State

What currently exists.

Example:

Backend:
Node.js + Express

Database:
PostgreSQL

Authentication:
JWT

Current feature:
Payment integration

9.3 Tasks

What has been completed and what remains.

Example:

[x] Authentication API
[x] Login UI
[x] Database schema
[ ] Payment integration
[ ] Webhook verification

9.4 Constraints

Rules the AI must respect.

Example:

- Do not replace PostgreSQL.
- Do not expose API keys.
- Do not modify production environment variables.
- Keep authentication based on httpOnly cookies.

9.5 Discoveries

Useful information discovered while working.

Example:

The payment provider sends webhook events
using POST /webhooks/payment.

Webhook signature verification is required.

9.6 Rejected Approaches

This is particularly important.

Example:

Rejected:

MongoDB

Reason:
The current application requires relational
transaction handling between multiple entities.

Status:
Rejected

This prevents another model from repeatedly suggesting the same approach.

---

10. Proposed Folder Structure

The initial project structure should be simple.

```
project/
│
├── src/
│   ├── ...
│
├── docs/
│   │
│   ├── memory/
│   │   │
│   │   ├── decisions/
│   │   │   ├── database.md
│   │   │   ├── authentication.md
│   │   │   └── architecture.md
│   │   │
│   │   ├── current-state.md
│   │   ├── tasks.md
│   │   ├── constraints.md
│   │   ├── discoveries.md
│   │   └── rejected.md
│   │
│   └── README.md
│
├── .ai-memory/
│   ├── config.json
│   └── index.json
│
├── .gitignore
└── package.json
```

---

11. Why Separate `/docs/memory` and `/.ai-memory`?

The distinction is important.

`/docs/memory`

Human-readable project knowledge.

Example:

docs/memory/decisions/database.md

This can be:

- Read by developers
- Committed to Git
- Reviewed in pull requests
- Edited manually
- Shared with other AI tools

`/.ai-memory`

Internal system metadata.

Example:

```
.ai-memory/
├── config.json
└── index.json
```

This can contain:

- Memory IDs
- Timestamps
- Hashes
- Embeddings
- Retrieval metadata
- Memory versions
- Relationships

This directory can optionally be ignored by Git.

---

12. Example Memory File

`docs/memory/decisions/database.md`

```
# Database Decision

## Decision

Use PostgreSQL as the primary database.

## Reason

The application contains strongly related entities
and requires transactional operations between
users, payments and subscriptions.

## Alternatives Considered

- MongoDB

## Rejected Alternatives

MongoDB was rejected for the current architecture
because the data model relies heavily on relational
relationships and transactional operations.

## Status

Active

## Created

2026-09-22
```

---

13. Example Current State

`docs/memory/current-state.md`

Current Project State

Stack

Frontend:
Next.js

Backend:
Node.js + Express

Database:
PostgreSQL

Cache:
Redis

Authentication:
JWT + httpOnly cookies

Current Feature

Payment integration.

Current Progress

Authentication is complete.

Database schema is complete.

Payment API is partially implemented.

Webhook verification remains unfinished.

Last Updated

2026-09-22

---

14. Example Tasks

`docs/memory/tasks.md`

Tasks

Completed

- [x] Create database schema
- [x] Create authentication API
- [x] Create login UI

In Progress

- [ ] Payment integration

Pending

- [ ] Webhook verification
- [ ] Payment failure handling
- [ ] Integration tests

---

15. Example Constraints

`docs/memory/constraints.md`

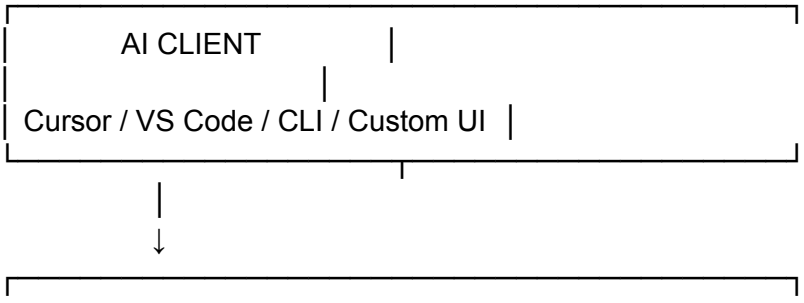
Project Constraints

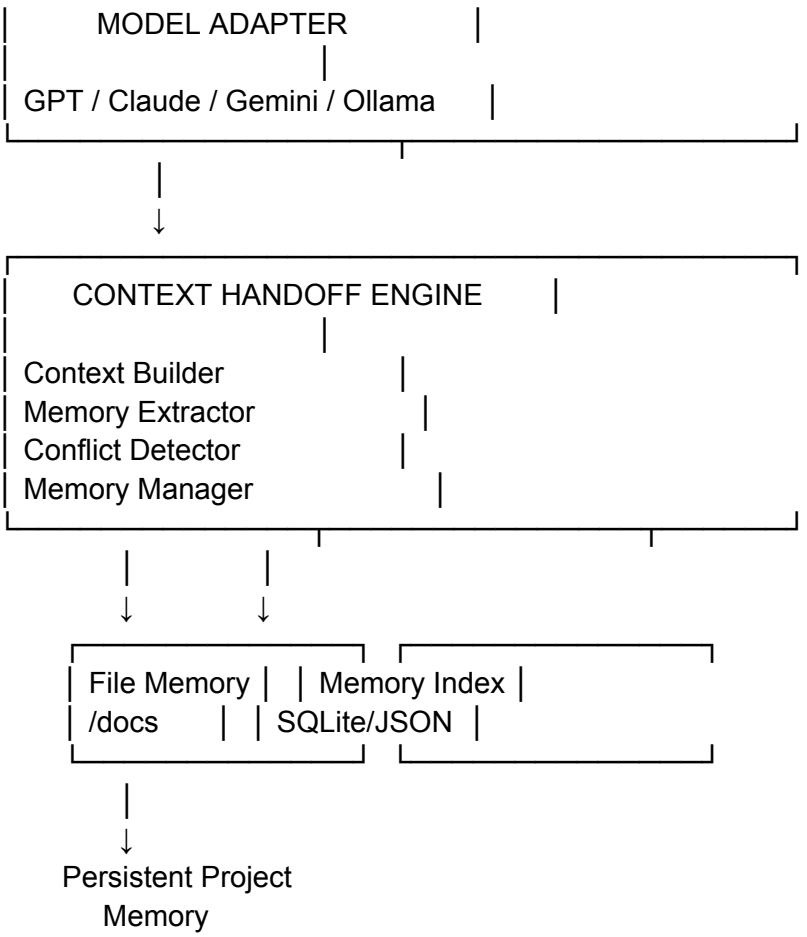
- Do not replace PostgreSQL.
- Do not change authentication architecture
  without explicit user approval.
- Never expose API keys.
- Do not modify production environment variables.
- Keep the existing API structure unless necessary.

---

16. System Architecture

The complete system can be divided into six components.





---

17. Component 1 — Model Adapter

The model adapter provides a common interface for different models.

Instead of the memory system knowing how every provider works:

GPT
Claude
Gemini
Ollama
OpenRouter

the system uses:

ModelAdapter

Example interface:

```
interface ModelAdapter {
   generate(request: ModelRequest): Promise<ModelResponse>;
}
```

Implementations:

OpenAIAdapter
AnthropicAdapter
GeminiAdapter
OllamaAdapter

This makes the memory system model-independent.

---

18. Component 2 — Memory Extractor

After an AI completes a meaningful operation, the system asks an extraction model:

> Identify only information that should persist beyond this conversation.

The extractor looks for:

Decision
Reason
Constraint
Task
Discovery
Rejected approach
Current state

Example input:

Conversation:
"We decided to use Redis because..."

Extractor output:

```
{
  "type": "decision",
  "title": "Caching",
  "decision": "Use Redis",
  "reason": "Reduce database load",
  "status": "active"
```

}

---

19. Component 3 — Memory Manager

The Memory Manager decides where information belongs.

For example:

type = decision
↓
docs/memory/decisions/

type = task
↓
docs/memory/tasks.md

type = constraint
↓
docs/memory/constraints.md

It also handles:

- Creating files
- Updating files
- Versioning
- Detecting conflicts
- Marking old decisions as superseded

---

20. Component 4 — Memory Retriever

When a new AI request arrives, the system determines which memories are relevant.

Example:

User:

Implement payment webhook verification.

The retriever may select:

current-state.md
tasks.md
decisions/payment.md
constraints.md

It does not need to load:

decisions/frontend.md
decisions/theme.md
decisions/database-indexing.md

unless they are relevant.

---

21. Component 5 — Context Builder

The Context Builder creates the prompt sent to the new AI.

Conceptually:

SYSTEM PROMPT

+

RELEVANT PROJECT MEMORY

+

RELEVANT SOURCE FILES

+

CURRENT USER REQUEST

Example:

PROJECT MEMORY

Active decisions:
- PostgreSQL is the primary database.
- JWT uses httpOnly cookies.
- Redis is used for caching.

Current state:
Payment API is partially implemented.

Constraints:
Do not change authentication.

Current task:
Implement webhook verification.

USER REQUEST:
Finish the payment webhook implementation.

The model can now continue with the correct context.

---

22. Component 6 — Conflict Detector

This is one of the most important components.

Suppose memory says:

Database:
PostgreSQL

A new model proposes:

Switch to MongoDB.

The system should not silently replace:

PostgreSQL

with:

MongoDB

Instead:

CONFLICT DETECTED

Existing decision:
PostgreSQL

New proposal:
MongoDB

Reason:
Model proposes document-based storage.

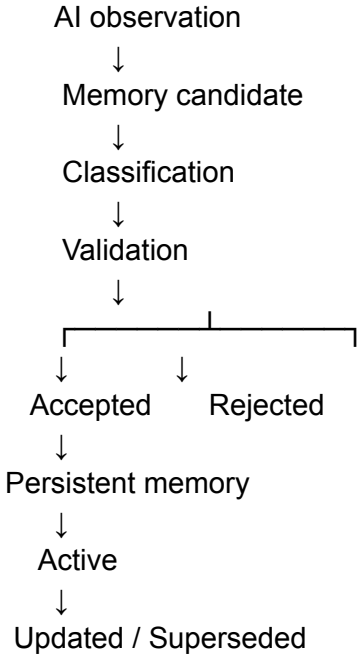
Action required:
User approval

The user can then decide.

---

23. Memory Lifecycle

Every memory should follow a lifecycle.



---

24. Memory States

Each decision can have:

proposed
active

superseded
rejected
archived

Example:

PostgreSQL
status: superseded

MongoDB
status: proposed

This gives the system historical awareness.

---

25. How the Files Are Created

When initializing a project:

ai-memory init

The system creates:

```
docs/
└── memory/
    ├── decisions/
    ├── current-state.md
    ├── tasks.md
    ├── constraints.md
    ├── discoveries.md
    └── rejected.md

.ai-memory/
├── config.json
└── index.json
```

The system should **not** immediately fill these files with fake information.

Instead:

ai-memory init

creates the structure.

Then:

ai-memory capture

creates memory from an existing AI session.

---

26. Example CLI

The first version could expose:

ai-memory init

Initialize project memory.

ai-memory capture

Extract persistent information from the current session.

ai-memory status

Show current memory state.

ai-memory search "authentication"

Search project memory.

ai-memory context "implement payment webhook"

Generate relevant context for an AI model.

ai-memory decisions

List architectural decisions.

ai-memory conflicts

Show conflicting decisions.

ai-memory update

Manually update project memory.

---

27. Proposed Backend Structure

A possible implementation repository:

```
ai-context-handoff/
│
├── src/
│   │
│   ├── adapters/
│   │   ├── openai.ts
│   │   ├── anthropic.ts
│   │   ├── gemini.ts
│   │   └── ollama.ts
│   │
│   ├── memory/
│   │   ├── extractor.ts
│   │   ├── manager.ts
│   │   ├── retriever.ts
│   │   ├── validator.ts
│   │   ├── conflict-detector.ts
│   │   └── indexer.ts
│   │
│   ├── context/
│   │   ├── builder.ts
│   │   ├── selector.ts
│   │   └── formatter.ts
│   │
│   ├── storage/
│   │   ├── filesystem.ts
│   │   ├── sqlite.ts
│   │   └── json.ts
│   │
│   ├── cli/
│   │   ├── init.ts
│   │   ├── capture.ts
│   │   ├── search.ts
│   │   ├── status.ts
│   │   └── context.ts
│   │
│   ├── types/
```

```
│   │   ├── memory.ts
│   │   ├── decision.ts
│   │   └── context.ts
│   │
│   └── index.ts
│
├── tests/
│   ├── memory/
│   ├── context/
│   ├── conflicts/
│   └── adapters/
│
├── docs/
│   ├── architecture.md
│   ├── design.md
│   └── protocol.md
│
├── package.json
├── README.md
└── LICENSE
```

---

28. Internal Data Model

Even though the primary memory is human-readable Markdown, the system should maintain metadata.

Example:

```
{
  "id": "decision-db-001",
  "type": "decision",
  "title": "Database",
  "status": "active",
  "file": "docs/memory/decisions/database.md",
  "createdAt": "2026-09-22",
  "updatedAt": "2026-09-22",
  "tags": [
    "database",
    "architecture"
  ]
}
```

This allows efficient searching without forcing the AI to read every file.

---

29. Initial Storage Strategy

The MVP should use:

Markdown
+
JSON index

rather than immediately introducing a vector database.

Why?

Because the first problem is:

> Can we reliably preserve and transfer project decisions?

not:

> Can we build a sophisticated semantic search engine?

The simplest architecture is easier to debug.

---

30. Later Storage Strategy

Once the system grows, it can support:

Markdown
+
SQLite
+
Embeddings
+
Vector search

For example:

docs/
↓
Indexer
↓
SQLite

↓
Embedding
↓
Vector index

Then a request such as:

"Why are we using Redis?"

can retrieve the relevant decision even if the exact words are different.

---

31. Local-First Architecture

A major feature should be **local-first operation**.

The user should be able to run:

AI Model
↓
Local Memory Engine
↓
Local Files

without sending project memory to an external service.

For example:

Ollama
↓
Memory Extractor
↓
Local filesystem
↓
SQLite

Cloud APIs can be optional.

This is particularly useful for:

- Private repositories
- Enterprise code
- Local development

- Security-sensitive projects

---

32. Integration Modes

The project can eventually support several integration methods.

Mode 1 — CLI

Terminal
↓
ai-memory

Simplest MVP.

Mode 2 — MCP

The memory engine can expose tools such as:

memory_search
memory_get
memory_create
memory_update
memory_decisions
memory_tasks

An AI agent can then interact with memory directly.

Mode 3 — IDE Extension

Example:

VS Code
↓
AI Context Handoff Extension
↓
Memory Engine

The extension could automatically capture decisions after an agent finishes a task.

Mode 4 — API

Expose:

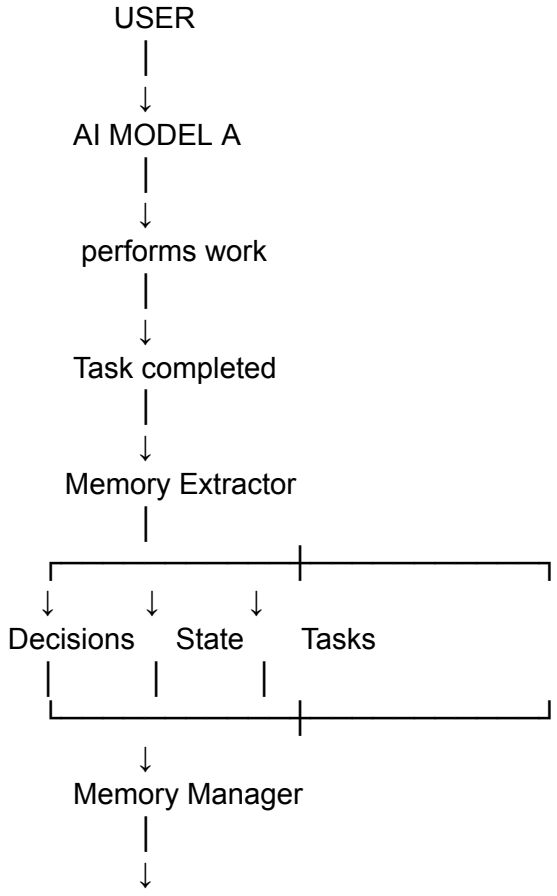
```
POST /memory/capture
GET  /memory/context
GET  /memory/search
POST /memory/decision
POST /memory/conflict
```

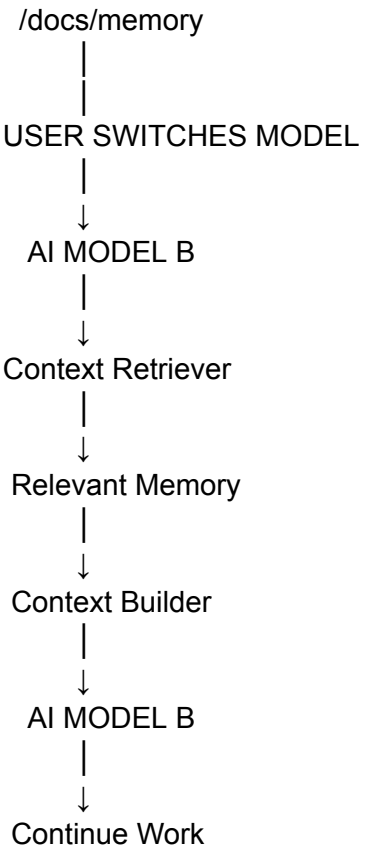
This allows other AI applications to integrate with it.

---

33. Model Handoff Workflow

The complete workflow would look like this:





---

34. Example End-to-End Scenario

Consider an application called:

MobiMart

Step 1

Developer asks Claude:

Implement authentication.

Claude determines:

JWT + httpOnly cookies

Memory system extracts:

Decision:

Use JWT with httpOnly cookies.

Reason:
Avoid exposing authentication tokens to client-side
JavaScript while maintaining stateless authentication.

Stored at:

docs/memory/decisions/authentication.md

Step 2

Developer switches to GPT.

User asks:

Continue implementing the backend.

Memory engine retrieves:

authentication.md
current-state.md
tasks.md
constraints.md

GPT receives:

Project Memory:

Authentication:
JWT + httpOnly cookies.

Current state:
Login API complete.
Registration API complete.
Refresh-token flow unfinished.

Task:
Implement refresh-token flow.

GPT continues.

Step 3

Developer switches to local Qwen.

Qwen doesn't need the complete Claude/GPT conversation.

It receives:

Relevant project memory:
~2,500 tokens

instead of potentially:

50,000+ tokens

The project can continue regardless of which model is being used.

---

35. Why This Is Different From Simple Chat History

Chat history answers:

> What did the AI say?

Project memory answers:

> What does the project currently believe to be true?

This distinction is extremely important.

Chat history

Conversation

Project memory

Decisions
+
Reasons
+
Constraints
+
Current state

+
Tasks
+
Discoveries
+
Rejected approaches

The second representation is much more useful for long-running software projects.

---

36. Handling Model Switching

When a model is changed:

Model A
↓
Current conversation
↓
Memory extraction
↓
Persistent memory
↓
Model B

The new model should receive:

Relevant memory
+
Recent conversation
+
Current task

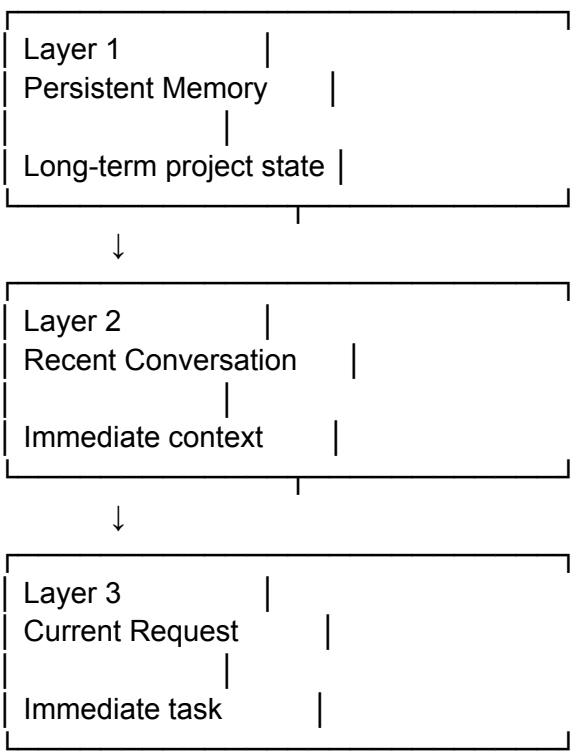
not necessarily the entire historical conversation.

This creates a hybrid context strategy.

---

37. Hybrid Context Strategy

The system should use three layers:



This is preferable to relying exclusively on either memory or conversation.

---

38. Memory Retrieval Strategy

Initially:

Keyword matching
+
file metadata
+
tags

Later:

Semantic search
+
embeddings
+
recency
+
importance
+

dependency

A memory score could eventually consider:

relevance
+
importance
+
recency
+
project dependency

For example:

Payment request

payment.md → highly relevant
database.md → relevant
frontend-theme.md → irrelevant

Only the first two should enter the context.

---

39. Memory Importance

Not all memories should be treated equally.

Example:

Importance 10:
Architecture decision

Importance 9:
Security constraint

Importance 8:
Database decision

Importance 5:
Implementation discovery

Importance 2:
Temporary debugging observation

Temporary information can eventually expire.

---

40. Memory Expiration

Some memories should not remain forever.

Example:

Temporary:
"Payment API currently returns 500."

Once fixed:

Status:
resolved

Permanent decisions:

Database = PostgreSQL

remain active until explicitly superseded.

---

41. Git Integration

The memory system should work naturally with Git.

Example:

git diff

could show:

docs/memory/decisions/database.md

A developer can review AI-generated memory changes before committing them.

This is important because project memory is effectively part of the project's documentation.

---

42. Safety Mechanism

The system should distinguish:

FACT
DECISION
PROPOSAL
ASSUMPTION
DISCOVERY

For example:

FACT:
PostgreSQL is currently configured.

DECISION:
PostgreSQL was selected for transactional requirements.

PROPOSAL:
Consider Redis for caching.

ASSUMPTION:
Redis is available in production.

This prevents an AI-generated assumption from being treated as project truth.

---

43. Proposed MVP

The first version should be deliberately small.

MVP Features

1. ai-memory init

2. Markdown memory storage

3. Memory extraction

4. Decision storage

5. Current-state storage

6. Task storage

7. Constraint storage

8. Memory search

9. Context generation

10. CLI interface

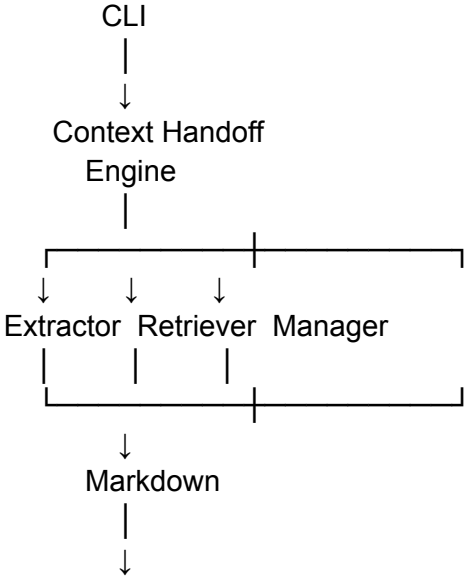
Do **not** start with:

IDE plugin
Multi-agent orchestration
Vector database
Cloud dashboard
Complex UI
Dozens of providers

Those can come later.

---

44. MVP Architecture



/docs/memory

---

45. Development Phases

Phase 1 — File-Based Memory

Implement:

ai-memory init
ai-memory status
ai-memory search

Create the memory folder structure.

Phase 2 — Memory Extraction

Implement:

ai-memory capture

Give an AI conversation to the extractor.

Extract:

decisions
tasks
constraints
state
discoveries

Phase 3 — Context Generation

Implement:

ai-memory context "<request>"

Return:

Relevant project memory

Phase 4 — Model Adapters

Support:

OpenAI
Anthropic
Gemini
Ollama

Phase 5 — Automatic Handoff

When the user changes model:

Old model
↓
capture memory
↓
new model
↓
inject memory

Phase 6 — Conflict Detection

Detect:

Existing decision
vs
new model proposal

Ask for user confirmation.

Phase 7 — MCP / IDE Integration

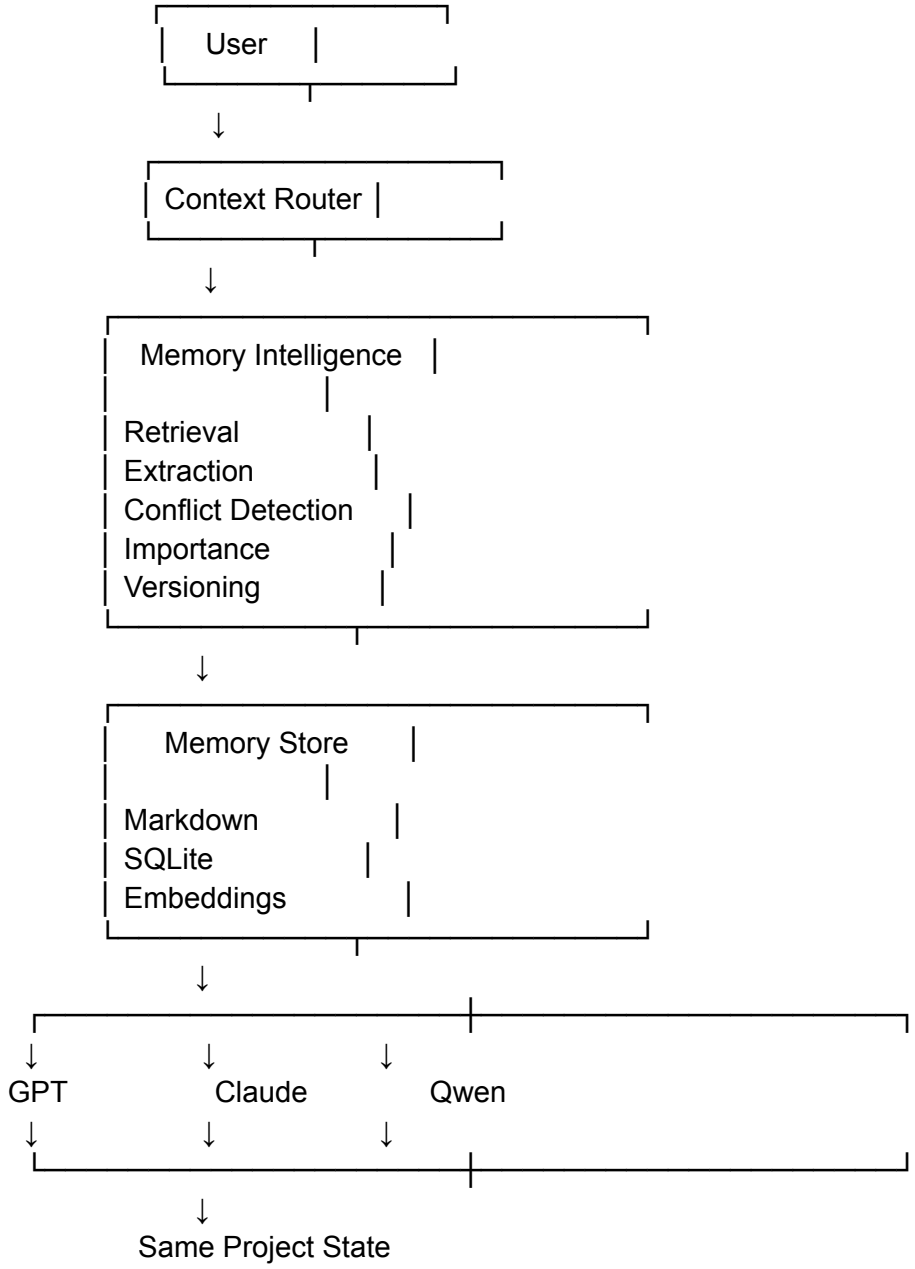
Expose memory to:

Cursor
VS Code
Claude Code
other MCP-compatible agents

---

46. Future Architecture

The long-term architecture could become:



---

47. Long-Term Vision

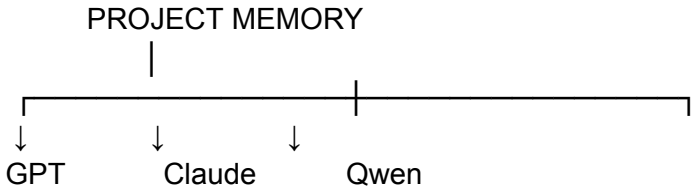
The ultimate goal is not to create another AI assistant.

The goal is to create a **shared memory protocol for AI agents**.

Today:

GPT has context
Claude has context
Cursor has context
Qwen has context

The proposed system changes this to:



The model becomes replaceable.

The project knowledge remains.

---

48. Core Principle

The most important principle of the system is:

> **Models are temporary workers. Project memory is persistent.**

A developer should be able to replace:

GPT

with:

Claude

or:

Qwen

without losing the important knowledge accumulated during development.

---

49. Expected Benefits

For developers

- Easier model switching
- Less repeated explanation
- Better continuity
- Better long-running projects
- Human-readable project history

For AI agents

- Smaller context requirements
- Better awareness of previous decisions
- Reduced repeated mistakes
- Better multi-agent collaboration

For local AI

- Smaller prompts
- Less VRAM pressure
- Better performance
- Easier use of smaller models

For teams

- Shared architectural knowledge
- AI-readable project documentation
- Decision history
- Better agent collaboration

---

50. Example Final Project Structure

A real project using the system could eventually look like:

my-project/
│

```
├── src/
│   ├── frontend/
│   └── backend/
│
├── tests/
│
├── docs/
│   │
│   ├── memory/
│   │   │
│   │   ├── decisions/
│   │   │   ├── authentication.md
│   │   │   ├── database.md
│   │   │   ├── architecture.md
│   │   │   └── caching.md
│   │   │
│   │   ├── current-state.md
│   │   ├── tasks.md
│   │   ├── constraints.md
│   │   ├── discoveries.md
│   │   └── rejected.md
│   │
│   └── README.md
│
├── .ai-memory/
│   ├── config.json
│   ├── index.json
│   └── cache/
│
├── .gitignore
├── package.json
└── README.md
```

---

51. Example User Experience

The ideal user experience should eventually be almost invisible.

The developer simply does:

User:
Implement authentication.

AI:

Done.

Authentication uses JWT with httpOnly cookies.

Memory layer automatically records:

Decision:
JWT + httpOnly cookies

Reason:
...

Later:

User:
Switch to Claude.

The system automatically:

Capture current memory
↓
Build context
↓
Start Claude
↓
Inject relevant memory

Claude immediately understands:

Authentication is already implemented.

Current architecture:
JWT + httpOnly cookies.

Do not replace this unless the user requests it.

The user does not need to explain everything again.

---

52. Summary

AI coding systems are becoming increasingly multi-model.

Users may switch between:

GPT
Claude
Gemini
Qwen
Local models

However, the project's **important decisions and state should not be tied to one model or one conversation.**

The proposed AI Context Handoff Layer introduces a persistent memory layer that stores:

Decisions
Reasons
Current State
Tasks
Constraints
Discoveries
Rejected Approaches

inside a structured project memory.

The system then retrieves the relevant information whenever another AI model begins working.

The architecture is:

AI Model
↓
Memory Extraction
↓
Persistent Project Memory
↓
Memory Retrieval
↓
Context Builder
↓
Another AI Model

The initial implementation should remain simple:

Markdown
+
JSON index
+

CLI
+
LLM extraction

and can later evolve into:

SQLite
+
Semantic Search
+
Embeddings
+
MCP
+
IDE integrations
+
Multi-agent memory

The fundamental idea is:

> **Don't make every AI remember the entire conversation. Make the project remember what matters.**

That allows AI models to become interchangeable workers while the project's knowledge, decisions, and state remain persistent and model-independent.